

Concurrency Control in DBMS

Concurrency Control

Concurrency Control in Database Management System is a procedure of managing simultaneous operations without conflicting with each other. It ensures that Database transactions are performed concurrently and accurately to produce correct results without violating data integrity of the respective Database.

- Reasons for using Concurrency control method in DBMS:
 - To apply Isolation through mutual exclusion between conflicting transactions
 - To resolve read-write and write-write conflict issues
 - To preserve database consistency through constantly preserving execution obstructions
 - The system needs to control the interaction among the concurrent transactions. This control is achieved using concurrent-control schemes.
 - Concurrency control helps to ensure serializability

Why we need concurrency control

Without Concurrency Control, problems may occur with concurrent transactions:

- **Lost Update Problem.**

Occurs when two transactions update the same data item, but both read the same original value before update (Figure 21.3(a), next slide)

- **The Temporary Update (or Dirty Read) Problem.**

This occurs when one transaction T1 updates a database item X, which is accessed (read) by another transaction T2; then T1 fails for some reason (Figure 21.3(b)); X was (read) by T2 before its value is changed back (rolled back or UNDONE) after T1 fails

(a)

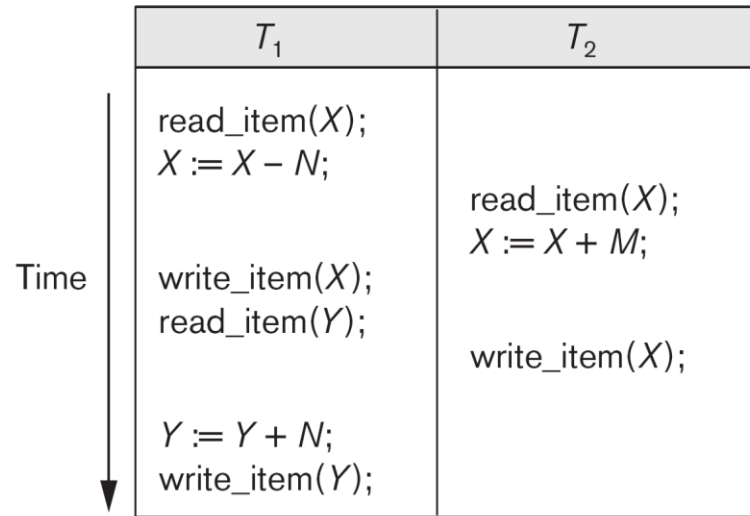
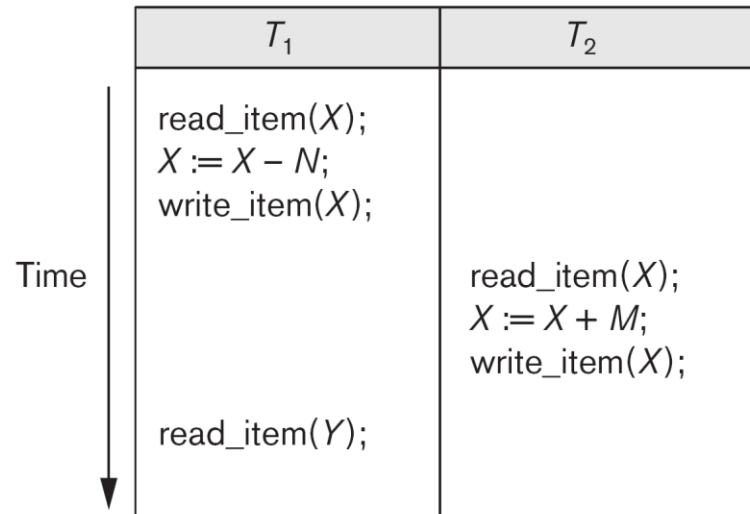


Figure 21.3

Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.

← Item X has an incorrect value because its update by T_1 is *lost* (overwritten).

(b)



← Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the *temporary* incorrect value of X .

Why we need concurrency control (cont.)

- **The Incorrect Summary Problem .**

One transaction is calculating an aggregate summary function on a number of records (for example, sum (total) of all bank account balances) while other transactions are updating some of these records (for example, transferring a large amount between two accounts, see Figure 21.3(c)); the aggregate function may read some values before they are updated and others after they are updated.

(c)

T_1	T_3
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>	<pre>sum := 0; read_item(A); sum := sum + A; ⋮ ⋮ ⋮ read_item(X); sum := sum + X; read_item(Y); sum := sum + Y;</pre>

← T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).

Why we need concurrency control (cont.)

- **The Unrepeatable Read Problem .**

A transaction T1 may read an item (say, available seats on a flight); later, T1 may read the same item again and get a different value because another transaction T2 has updated the item (reserved seats on the flight) between the two reads by T1

Example:

T1	T2
Read(X)	
	Read(X)
Write(X)	
	Read(X)

Concurrency Control Protocols

Different concurrency control protocols offer different benefits between the amount of concurrency they allow and the amount of overhead that they impose.

Following are the Concurrency Control techniques in DBMS:

- Lock-Based Protocols (in Syllabus)
- Two Phase Locking Protocol (in syllabus)
- Timestamp-Based Protocols (not in Syllabus)
- Validation-Based Protocols (not in syllabus)
- Graph-based Protocols (not in syllabus)

Lock-based Protocols

Lock Based Protocols in DBMS is a mechanism in which a transaction cannot Read or Write the data until it acquires an appropriate lock.

Lock based protocols help to eliminate the concurrency problem in DBMS for simultaneous transactions by locking or isolating a particular transaction to a single user.

A lock is a data variable which is associated with a data item. This lock signifies that operations that can be performed on the data item.

Locks in DBMS help synchronize access to the database items by concurrent transactions.

All lock requests are made to the concurrency-control manager. Transactions proceed only once the lock request is granted.

TYPES OF LOCKS

- **Binary Locks:** A Binary lock on a data item can either be locked or unlocked.
- **Shared/exclusive:** This type of locking mechanism separates the locks in DBMS based on their uses. If a lock is acquired on a data item to perform a write operation, it is called an exclusive lock.

TYPES OF LOCKS cont...

1.Shared Lock (S):

A shared lock is also called a **Read-only lock**. With the shared lock, the data item can be shared between transactions. This is because you will never have permission to update data on the data item.

For example, consider a case where two transactions are reading the account balance of a person. The database will let them read by placing a shared lock. However, if another transaction wants to update that account's balance, shared lock prevent it until the reading process is over.

TYPES OF LOCKS cont...

2. Exclusive Lock (X):

With the Exclusive Lock, a data item can be read as well as written. This is exclusive and can't be held concurrently on the same data item. X-lock is requested using lock-x instruction. Transactions may unlock the data item after finishing the 'write' operation.

For example, when a transaction needs to update the account balance of a person. You can allow this transaction by placing X lock on it. Therefore, when the second transaction wants to read or write, exclusive lock prevents this operation.

TYPES OF LOCKS cont...

T1

S(A)

R(A)

U(A)

T2

X(A)

R(A)

W(A)

U(A)

Lock-Based Protocols (Cont.)

- **Lock-compatibility matrix**

	S	X
S	true	false
X	false	false

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- Any number of transactions can hold shared locks on an item,
 - but if any transaction holds an exclusive lock on the item no other transaction may hold any lock on the item.
- If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

Lock-Based Protocols (Cont.)

- Example of a transaction performing locking:

```
 $T_2$ : lock-S(A);  
      read (A);  
      unlock(A);  
      lock-S(B);  
      read (B);  
      unlock(B);  
      display(A+B)
```

- A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules.

Pitfalls of Lock-Based Protocols

- May not free from Irrecoverability
- May not free from deadlock
- May not free from Starvation

Pitfalls of Lock-Based Protocols (May not free from Irrecoverability)

1.	T1	T2	
	X(A)		
	R(A)		
	W(A)		
	U(A)		
		S(A)	
		R(A)	Dirty read problem
		U(A)	
		commit	
	Abort		

Pitfalls of Lock-Based Protocols(Deadlock)

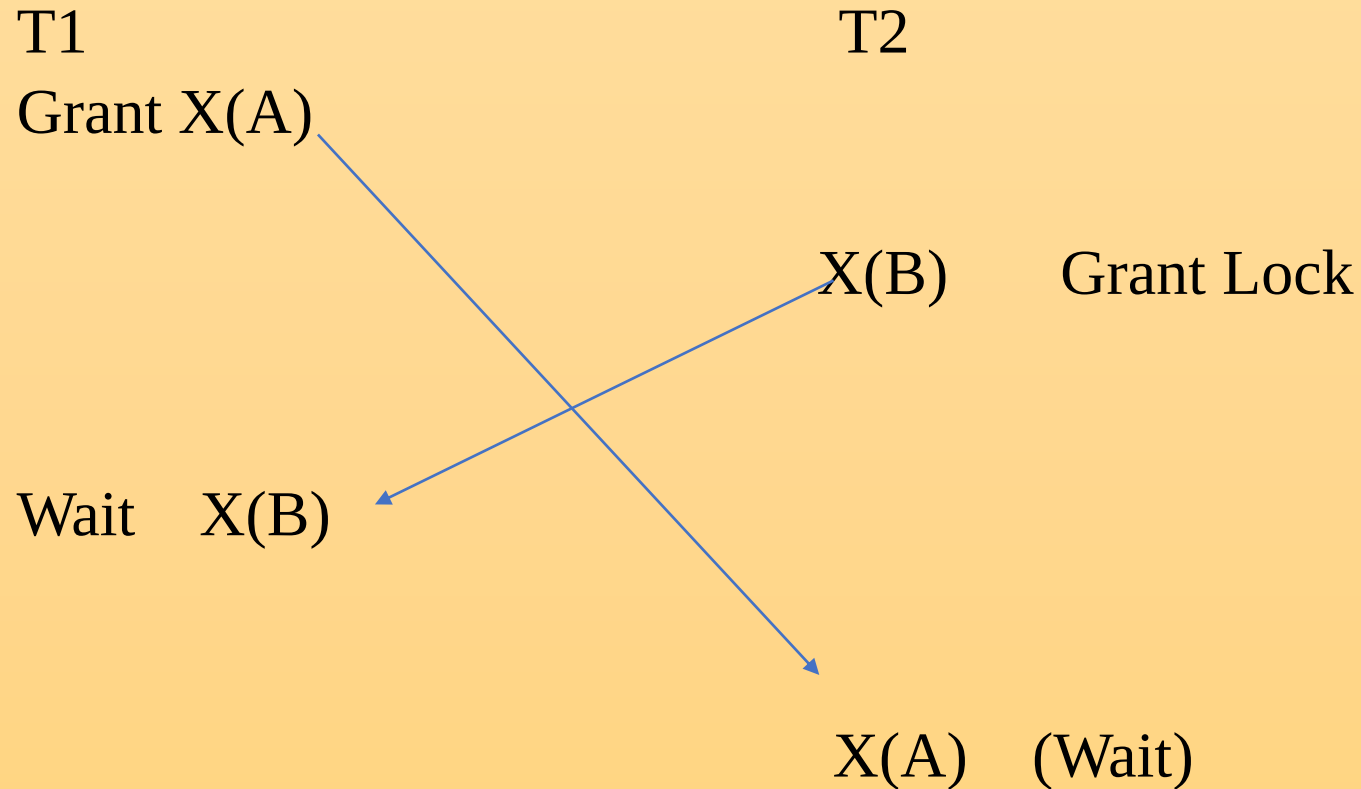
T_3	T_4
lock-x (B) read (B) $B := B - 50$ write (B) lock-x (A)	lock-s (A) read (A) lock-s (B)

Neither T_3 nor T_4 can make progress — executing **lock-S(B)** causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X(A)** causes T_3 to wait for T_4 to release its lock on A . Such a situation is called a **deadlock**.

- To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

Pitfalls of Lock-Based Protocols(Deadlock)

• Consider the partial schedule



Deadlock

Deadlock refers to a specific situation where two or more processes are waiting for each other to release a resource or more than two processes are waiting for the resource in a circular chain.

Pitfalls of Lock-Based Protocols (Cont.)

- The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil.
- **Starvation** is also possible if concurrency control manager is badly designed. For example:
 - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
 - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.

concepts

Starvation

Starvation is the situation when a transaction needs to wait for an indefinite period to acquire a lock.

Following are the reasons for Starvation:

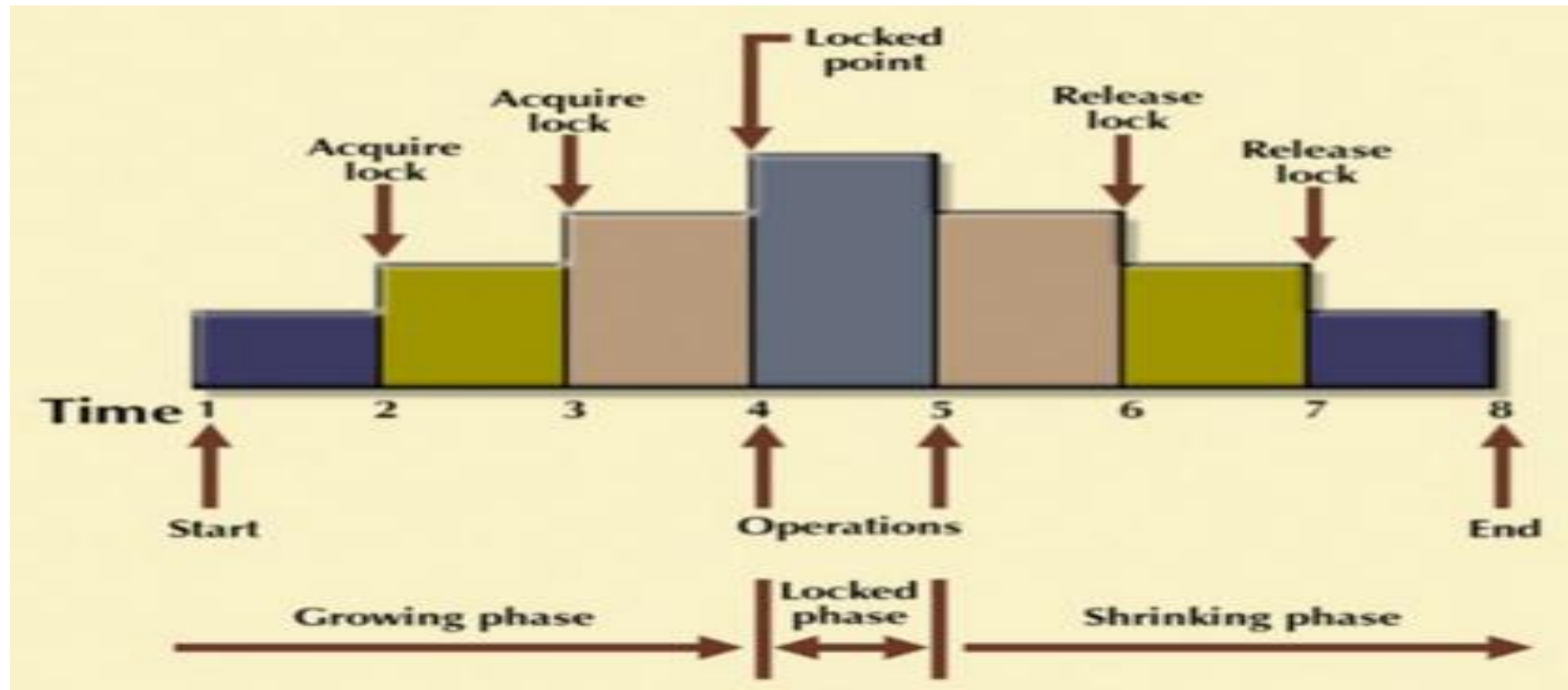
- When waiting scheme for locked items is not properly managed
- In the case of resource leak, the same transaction is selected as a victim repeatedly

Two Phase Locking Protocol(2PL)

It is a method of concurrency control in DBMS that ensures serializability by applying a lock to the transaction data which blocks other transactions to access the same data simultaneously. Two Phase Locking protocol helps to eliminate the concurrency problem in DBMS. This locking protocol divides the execution phase of a transaction into three different parts.

- In the first part, when the transaction begins to execute, it requires permission for the locks it needs.
- The second part is where the transaction obtains all the locks. When a transaction releases its first lock, the third phase starts.
- In this third parts, the transaction cannot demand any new locks. Instead, it only releases the acquired locks.

Two Phase Locking Protocol



Two Phase Locking Protocol

The Two-Phase Locking protocol allows each transaction to make a lock or unlock request in two steps:

- Growing Phase:** In this phase transaction may obtain locks but may not release any locks.
- Shrinking Phase:** In this phase, a transaction may release locks but not obtain any new lock

It is true that the 2PL protocol offers serializability. However, it does not ensure that deadlocks do not happen.

Two Phase Locking Protocol

	T_1	T_2
1	lock-S(A)	
2		lock-S(A)
3	lock-X(B)	
4		
5	Unlock(A)	
6		Lock-X(C)
7	Unlock(B)	
8		Unlock(A)
9		Unlock(C)
10		

Transaction T_1 :

- The growing Phase is from steps 1-3.
- The shrinking Phase is from steps 5-7.
- Lock Point at 3

Transaction T_2 :

- The growing Phase is from steps 2-6.
- The shrinking Phase is from steps 8-9.
- Lock Point at 6

Two Phase Locking Protocol(Cascading Rollbacks in 2-PL

To analyze the schedule. because of **Dirty Read** in T_2 and T_3 in lines 8 and 12 respectively, when T_1 failed we have to roll back others also. Hence, **Cascading Rollbacks** are possible in 2-PL.

	T_1	T_2	T_3
1	Lock-X(A)		
2	Read(A)		
3	Write(A)		
4	Lock-S(B) --->LP	Rollback	
5	Read(B)		Rollback
6	Unlock(A),Unlock(B)		
7		Lock-X(A) ----->LP	
8		Read(A)	
9		Write(A)	
10		Unlock(A)	
11			Lock-S(A) ----->LP
12			Read(A)

FAIL_____Rollback

LP - Lock Point

Read(A) in T_2 and T_3 denotes Dirty Read because of **Write(A)** in T_1 .

The Two-Phase Locking Protocol (Cont.)

- Two-phase locking *does not* ensure freedom from deadlocks
- Cascading roll-back is possible under two-phase locking. To avoid this, follow a modified protocol called **strict two-phase locking**. Here a transaction must hold all its exclusive locks till it commits/aborts. **Recoverable and Cascadeless, but not free from Deadlock.**
- **Rigorous two-phase locking** is even stricter: here *all* locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit. **Recoverable and Cascadeless, but not free from Deadlock.**

Conservative two-Phase Locking

also called as Static Two – Phase Locking Protocol. This protocol is almost free from deadlocks as all required items are listed in advanced. It requires locking of all data items to access before the transaction starts. **It prevents deadlocks but not starvation and cascading rollbacks.** However, it is difficult to use in practice because of the need to predeclare the read-set and the write-set which is not possible in many situations. In practice, the most popular variation of 2-PL is Strict 2-PL. This protocol requires the transaction to lock all the items it access before the transaction begins execution by predeclaring its read-set and write-set. If any of the predeclared items needed cannot be locked, the transaction does not lock any of the items, instead, it waits until all the items are available for locking.

Strict 2PL	
T1	T2
s-lock(A)	
read(A)	
	s-lock(A)
x-lock(B)	
unlock(A)	
read(B)	
write(B)	
	read(A)
commit	
unlock(B)	
	s-lock(B)
	read(B)
	unlock(A)
	unlock(B)
	commit

Rigorous 2PL	
T1	T2
s-lock(A)	
read(A)	
	s-lock(A)
x-lock(B)	
	read(A)
read(B)	
write(B)	
commit	
unlock(B)	
	s-lock(B)
	read(B)
unlock(A)	
	commit
	unlock(A)
	unlock(B)

Strict 2-PL	Conservative 2-PL	Rigorous 2-PL
In Strict 2-PL, A transaction can acquire locks on data items whenever it requires (only in growing phase) during its execution.	A transaction has to acquire locks on all the data items it requires before the transaction begins its execution.	A transaction can acquire locks on data items whenever it requires (only in growing phase) during its execution.
It has growing phase.	It does not have a growing phase.	It has a growing phase.
It has partial shrinking phase.	It has a shrinking phase.	It does not have a shrinking phase.
It ensures that the schedule generated would be Serializable, Recoverable and Cascadeless.	It ensures that the schedule generated would be Serializable and Deadlock-Free.	It ensures that the schedule generated would be Serializable, Recoverable and Cascadeless.
It does not ensures Deadlock-Free schedule.	It does not ensures Recoverable and Cascadeless schedule.	It does not ensures Deadlock-Free schedule.
It ensures that the schedule generated would be Strict .	It does not ensure Strict Schedule.	It ensures that the schedule generated would be Strict .

Thank You